

Technical Architecture

Technology Stack

Layer	Technology
Frontend	React 19, TypeScript, Vite, Tailwind CSS, MSAL React, i18next, lucide-react, Application Insights Web SDK.
Backend API	FastAPI, Pydantic, SQLAlchemy, pyodbc, Alembic, PyJWT, MSAL, Azure SDKs, OpenTelemetry.
Workers	Python, Azure Service Bus SDK, DefaultAzureCredential, OpenTelemetry, pyodbc.
ML and analytics	pandas, numpy, scikit-learn, sentence-transformers, SHAP, networkx, Azure OpenAI.
Infrastructure	Terraform, Azure Container Apps, Azure Static Web Apps, Azure Front Door, Azure SQL, Azure Service Bus, Key Vault, Storage, Log Analytics, Application Insights.
CI/CD	GitHub Actions deploying frontend, backend, auxiliary service, integrity worker, payroll broker, fraud analytics job, and KQL validation assets.

Frontend Architecture

The frontend is a React/Vite application under **frontend**.

Key modules:

- **src/App.tsx**: main application shell and user workflows.
- **src/msalInstance.ts**, **src/authConfig.ts**: Microsoft identity configuration.
- **src/services/api.ts**: token acquisition and authenticated API helper.
- **src/contexts/TenantConfigContext.tsx**: tenant config loading, theming, currency, locale, wrong-domain detection.
- **src/contexts/ImpersonationContext.tsx**: admin impersonation state.
- **src/components/AnalyticsDashboard.tsx**: admin analytics, integrity findings, forecast, AI ask/investigate.
- **src/components/HRBPReviewTab.tsx**: HRBP review queue.
- **src/components/NominationLogsDrawer.tsx**: admin log access for nominations.
- **src/i18n**: English and Korean localization.

Frontend responsibilities:

- Sign users in through MSAL.
- Request API access tokens with the API scope.
- Call backend APIs with **Authorization: Bearer <token>**.
- Add **X-Impersonate-User** when admin impersonation is active.
- Load public tenant branding before login.
- Load authenticated tenant config after login.
- Apply CSS custom properties for tenant theming.
- Change i18next language based on tenant locale.
- Enforce role-gated tabs in the UI.

Backend Architecture

The backend is a FastAPI application under `backend`.

Main entry point:

- `backend/main.py`

Router modules:

- `demo_router.py`
- `hrbp_router.py`
- `users_router.py`
- `nominations_router.py`
- `admin_router.py`
- `analytics_router.py`
- `webhooks_router.py`
- `internal_router.py`
- `payroll_router.py`

Core backend services:

- `auth.py`: JWT validation, tenant resolution, role checks, impersonation.
- `utils/sqlhelper2.py`: database access and business operations.
- `utils/service_bus_publisher.py`: Service Bus event publishing with trace context.
- `utils/certificate.py`: certificate generation and SAS links.
- `utils/forecasting.py`: forecast response support.
- `fraud_ml.py`: live model cache and refresh management.
- `agents`: analytics ask/investigate agents and export helpers.

Startup behavior:

- Loads environment variables.
- Configures logging.
- Configures Azure Monitor OpenTelemetry when `APPLICATIONINSIGHTS_CONNECTION_STRING` is present.
- Verifies ORM-defined tables with `sqlhelper.create_all_tables()`.
- Starts a fraud model eviction loop controlled by `MODEL_EVICTION_INTERVAL_SECONDS` and `MODEL_IDLE_TTL_SECONDS`.

Authentication Architecture

Authentication is Microsoft Entra ID based.

Backend flow:

1. Read bearer token.
2. Decode token without verification to extract `tid`.
3. Resolve `tid` to internal `TenantId` through the `Tenants` table.
4. Optionally log domain mismatch based on Origin and tenant domain.
5. Fetch signing key from Microsoft JWKS.
6. Verify signature, audience, issuer, and expiry.
7. Extract UPN or email.
8. Resolve the user in the tenant roster.

9. Return user context containing `UserId`, `TenantId`, `AadTenantId`, UPN, profile fields, and roles.

Admin impersonation:

- Only users with `AWard_Nomination_Admin` or `Administrator` may impersonate.
- Impersonation is restricted to the same internal tenant.
- Impersonated users do not inherit admin roles.
- Impersonation actions are logged.

API Structure

The API is organized around:

- Public tenant branding.
- Current user and tenant config.
- Nomination lifecycle.
- HRBP review.
- Admin analytics and investigation.
- Admin operational endpoints.
- Demo self-registration.
- Payroll lookup.
- Internal callbacks and scheduled checks.
- Webhook endpoints.

See [14_API_Specification.md](#).

Data Access Architecture

`backend/utils/sqlhelper2.py` centralizes database operations. The code uses SQLAlchemy ORM declarations for selected tables and direct SQL for many query-heavy workflows.

Important patterns:

- Tenant ID is passed to most read paths.
- Nomination access is checked through approver or tenant joins.
- Analytics queries aggregate by tenant.
- Service Bus idempotency uses `ProcessedEvents`.
- Conversation state for AI analytics is stored in SQL.
- HRBP flags and labels are persisted for later review and model retraining.

Async/Event Architecture

The application uses Azure Service Bus as the event backbone.

Events include:

- `nomination.submitted`
- `nomination.created`
- `nomination.approved`
- `nomination.description-rejected`
- `nomination.fraud-flagged`

- `nomination.hrbp-approved`
- `nomination.hrbp-rejected`
- `nomination.hrbp-sla-breach`
- `notification.requested`
- `notification.access_requested`
- `payout.accepted`
- `payroll.accepted`
- `payroll.failed`

Each worker:

- Uses managed identity credentials.
- Receives from a subscription in Peek Lock mode.
- Completes messages on success.
- Abandons messages on transient failure.
- Dead-letters invalid JSON or unrecoverable messages where implemented.
- Uses `ProcessedEvents` for idempotent side effects.
- Preserves distributed trace context via Service Bus application properties.

AI/ML Architecture

AI and ML are split into:

- Submission-time integrity check: semantic checks and P2P fraud scoring.
- Weekly graph and model job: graph pattern detection, Random Forest retraining, historical scoring, forecasts.
- Admin AI analytics: ask/investigate agents, SQL querying, export generation, notifications.

Models are tenant-specific where fraud scoring is concerned. Model artifacts are stored in Blob Storage as `fraud_detection_model_tenant_<TenantId>.pkl`.

Observability Architecture

Observability uses:

- Structured JSON logging with an `App_Log` prefix in service loggers.
- Application Insights connection strings.
- OpenTelemetry FastAPI instrumentation.
- Trace propagation through Service Bus.
- Log Analytics KQL under `analytics/kusto-queries`.
- Grafana and executive dashboard documents under `Observability`.
- Health probes at `/health` and Front Door HEAD probes.

Security Architecture

Security relies on:

- Entra ID token verification.
- Entra app roles for admin access.
- Application roles in `UserRoles` for HRBP and PayrollBP.

- Tenant-scoped data access.
- Managed identity and RBAC for Azure resources.
- Key Vault-backed secrets.
- Service Bus local auth disabled in Terraform.
- Front Door WAF in prevention mode.
- TLS 1.2 minimum for Service Bus and Front Door custom domains.

See [09_Security_Architecture.md](#).